DESIGN DOCUMENT · LOTTERY SEARCH SYSTEM

Searching ten million tickets, and handing out each one exactly once

Search 10,000,000 six-digit lottery tickets by a wildcard pattern, and distribute matches to concurrent users so that **no ticket ever reaches two people**. Every quantitative claim below is a measurement taken from a working implementation under enforced resource limits, not an estimate.

01 METADATA

Document control

TITLE	Lottery Search System — Design Document
AUTHOR	Natthawat Narin, Senior Backend Developer
STATUS	Final — submitted for review
VERSION	1.0
DATE	2026-09-05
REVIEWERS	—
REPOSITORY	github.com/nthw-dev/challenge-lottery-search-system — this document, the decision records, the full results analysis and every evidence bundle behind the numbers
PROOF OF CONCEPT	github.com/nthw-dev/poc-lottery-search-system — the solution architecture, data structures, algorithms and load tests that produced every measurement here
RESPONDS TO	`challenge-lottery-search-system.md`
DECISION LOG	`adr/` — eight architecture decision records, indexed in Appendix A

Every quantitative claim in this document is a measurement, not an estimate. A proof of concept was built and load-tested on Kubernetes with enforced resource limits specifically to produce those numbers; §8 indexes the raw evidence, and §9 lists what

remains unproven.

02 CONTEXT AND SCOPE

The real search space is one million, not ten million

The challenge: search 10,000,000 six-digit lottery tickets by a 6-character pattern with `*` wildcards, and distribute matching tickets to concurrent users without ever handing the same ticket to two people. The deliverable is a design, with a recommended storage technology, an indexing strategy for wildcard matching, a performance analysis, and a concurrency strategy that prevents duplicate distribution.

2.1 The insight the design rests on

A six-digit ticket has only **10^6 = 1,000,000 possible values**. Ten million tickets therefore carry an average of ten tickets per distinct number.

This reframes the problem. The naive reading is "search ten million rows". The accurate reading is "decide which of one million *numbers* match, then fetch tickets for those numbers". The second search space is ten times smaller, it is fixed no matter how many tickets exist, and it is small enough to stay permanently in cache. Every significant decision below follows from this.

The second thing to understand before reading any performance number is that result-set size varies by six orders of magnitude with the number of wildcards.

PATTERN	WILDCARDS	NUMBERS MATCHED	TICKETS MATCHED
`123456`	0	1	~10
`123***`	3	1,000	~10,000
`****23`	4	10,000	~100,000
`**3***`	5	100,000	~1,000,000
`******`	6	1,000,000	10,000,000

A design that is fast only for narrow patterns, or only for wide ones, has not solved the problem. Performance must be reported per wildcard class, which is how §6.2 reports it.

A wildcard may appear in any position, so an ordinary B-tree on the ticket number helps only when the leading digits are fixed. `****23` cannot use it at all. That single fact is the technical core of the challenge.

03 GOALS AND NON-GOALS

Four testable goals, all met; six things deliberately left out

3.1 Goals

Each goal was stated as a testable hypothesis before the proof of concept was built, and each is paired here with what was measured.

GOAL	TARGET	MEASURED	RESULT
G1 Wildcard search in any position at 10M rows, under a 500m-CPU / 128Mi API pod	p95 < 100 ms for every wildcard class at 100 concurrent users	p95 55.8 ms, flat across classes, 5,346 req/s	**met (§6.2)**
G2 Many users searching the same pattern at once never receive the same ticket	zero duplicates under 200-way contention	351,481 allocations, 0 duplicates, 0 ledger/state mismatches	**met (§8.1)**
G3 The chosen index strategy beats a plain `LIKE` baseline by a wide margin at acceptable storage cost	≥ 10× on the hardest pattern; index size comparable to alternatives	four orders of magnitude on the exact pattern; 192 MB of indexes versus 397 MB for the per-digit alternative	**met (§6.2)**
G4 Stability under sustained mixed load within the pod's memory limit	no OOM kill, memory well under 128Mi across a 10-minute soak	743,994 requests, 0 errors, 0 restarts, peak 27 MiB	**met (§7.3)**

3.2 Non-goals

Deliberately out of scope, so that the design proves one database is sufficient before anything is added to it:

- Authentication and authorization

- Payment flow, prize drawing, and draw-period scheduling
- Horizontal scaling of the database, replication, read replicas
- Any caching layer — Redis is excluded on purpose
- A production observability stack (Prometheus, Grafana, tracing)
- CI/CD pipeline

04 OVERVIEW

One stateless API, one database, three tables

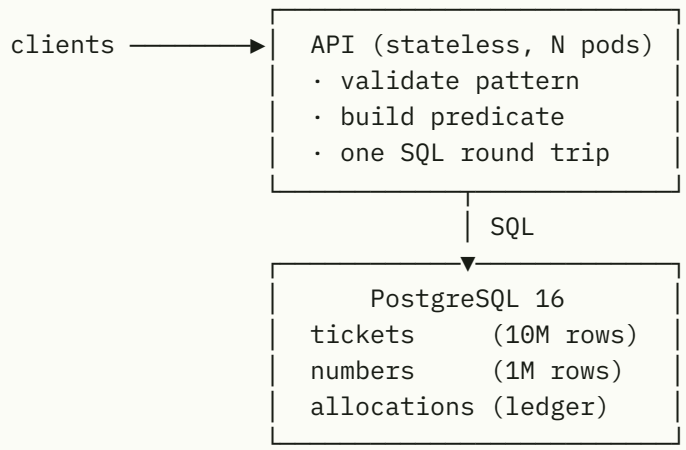
RECOMMENDED STORAGE	A single **PostgreSQL 16** instance — no cache, no lock service, no search engine
SEARCH METHOD	Resolve the pattern against a **1,000,000-row** number table, then join into tickets
DISTRIBUTION METHOD	**FOR UPDATE SKIP LOCKED** in one statement, with an append-only allocation ledger
MEASURED AT 10M ROWS	**p95 56 ms** at 5,346 req/s on one 500m-CPU pod; **351,481** concurrent allocations, **0** duplicates

The design is one stateless API in front of one PostgreSQL database holding three tables. A search first decides which of the one million possible numbers match the pattern, using a small static table indexed per digit, and only then touches the ten-million-row ticket table. An allocation is a single SQL statement whose row locks make it impossible for two users to receive the same ticket, with the outcome written to an audit ledger in the same transaction. Nothing else exists: no cache, no lock service, no search engine. Measured on PostgreSQL 16; recommended on 17 (§6.4).

05 DETAILED DESIGN

Architecture, data model, search, and concurrency

5.1 Architecture and API



There is deliberately nothing else. No cache tier, no queue, no lock service, no search engine. Every component that does not exist cannot fail, cannot drift out of sync with the database, and does not need to be operated. The concurrency guarantee the challenge asks for is provided by the database's own row locking, which removes the usual reason to introduce Redis.

The API holds no state, which is what makes horizontal scaling the answer to throughput.

ENDPOINT	PURPOSE
`POST /v1/search`	Preview matches, reserves nothing. Result limit capped at 100.
`POST /v1/allocate`	Atomically reserve matching tickets for a user. Limit capped at 20.
`POST /v1/release`	Return reserved tickets to the pool.
`GET /healthz`	Liveness. Process only — never touches the database (§7.3).
`GET /readyz`	Readiness. Pings the database.

A pattern must match `^[0-9*]{6}$` and is rejected at the handler before any database work. That is both a correctness rule and the reason the predicate builder can safely inline digits into SQL. Result counts are *estimated* as $10^{\text{wildcards}} \times$ tickets-per-number, never counted: running a real `COUNT(*)` for `**3***` would scan a million rows to produce a number nobody needs to be exact.

5.2 Data model

```
CREATE TABLE tickets (
  id          bigint    PRIMARY KEY,
  number      int       NOT NULL,            -- 0..999999
  status      smallint  NOT NULL DEFAULT 0,  -- 0=available, 1=reserved, 2=sold
  reserved_by int,
  reserved_at timestamptz
);

CREATE TABLE numbers (                       -- the 1M-row search space
  n  int PRIMARY KEY,                        -- 0..999999
  d1 smallint NOT NULL, d2 smallint NOT NULL, d3 smallint NOT NULL,
  d4 smallint NOT NULL, d5 smallint NOT NULL, d6 smallint NOT NULL
);

CREATE TABLE allocations (                   -- append-only ledger
  ticket_id bigint      NOT NULL,
  user_id   int         NOT NULL,
  at        timestamptz NOT NULL DEFAULT now()
);
```

DECISION	REASON
`number` is `int`, not `char(6)`	Saves roughly 60 MB at 10M rows, compares faster, and digits can be extracted arithmetically. Zero-padding is a presentation concern applied on output.
Digits are **not** stored on `tickets`	Six generated columns would add about 120 MB to the heap. The digit columns live on the 1M-row table instead, at a fraction of the cost.
`status` is `smallint`	Smaller and faster to compare than an enum or text, and sufficient for three states.
A separate `allocations` ledger	Makes the duplicate-distribution guarantee **auditable with one query** rather than inferred. This is the evidence behind §5.4.

`numbers` is a static lookup table: one million rows generated once and never written again. It is the reason the design absorbs more tickets without the search cost changing.

5.3 Search algorithm and indexes

For a pattern such as `1*3**6`:

1. **Validate**, then split the pattern into fixed digits and wildcards.
2. **Resolve the number space** against `numbers`. Two cases collapse to something cheaper than an index lookup: a fixed *leading* run (`123***`) becomes the primary-key range `n BETWEEN 123000 AND 123999`, and a fully specified pattern (`123456`) skips the table entirely as `number = 123456`. Everything else uses per-digit B-trees.
3. **Fetch tickets** by joining those numbers into `tickets(number)` through a partial index restricted to `status = 0`.
4. **Stop early** — the `LIMIT` is pushed into the scan.

```
-- 123***  → a primary-key range on the 1M-row table
SELECT id, number FROM tickets
WHERE status = 0
  AND number IN (SELECT n FROM numbers WHERE n BETWEEN 123000 AND 123999)
LIMIT 20;

-- ****23  → per-digit indexes on the 1M-row table
SELECT id, number FROM tickets
WHERE status = 0
  AND number IN (SELECT n FROM numbers WHERE d5 = 2 AND d6 = 3)
LIMIT 20;
```

Seven indexes in total — six digit indexes on `numbers` at about 6.8 MB each, plus a partial B-tree on `tickets(number)` — **192 MB** at 10M tickets. The `numbers` half is fixed forever; only the ticket index grows, and being partial on `status = 0` it *shrinks* as inventory sells.

TWO RULES THE MEASUREMENTS FORCED

No `ORDER BY` on search. The obvious `ORDER BY id LIMIT 20` turned out to be the largest single cost in the system: at 10M rows it makes the planner walk the primary key and probe the number table once per row, 52,000 probes and 16–424 ms for `123***`. Search results have no meaningful order, so it was removed and per-query time fell below a millisecond. **Never materialize the full match set** either — `**3***` matches a million tickets while the user wants twenty, so the `LIMIT` must reach the index scan for time-to-first-page to stay constant.

5.4 Concurrency and distribution

This is the requirement the challenge weights most heavily, and it is met by the database alone.

```
WITH picked AS (
  SELECT id FROM tickets
  WHERE status = 0 AND <pattern predicate>
  LIMIT $2
  FOR UPDATE SKIP LOCKED          -- the guarantee
), upd AS (
  UPDATE tickets t SET status = 1, reserved_by = $1, reserved_at = now()
  FROM picked WHERE t.id = picked.id
  RETURNING t.id, t.number
), ledger AS (
  INSERT INTO allocations (ticket_id, user_id) SELECT id, $1 FROM upd
)
SELECT id, number FROM upd;
```

One statement is one transaction, so selecting, reserving and recording either all happen or none do. `FOR UPDATE` locks the chosen rows; `SKIP LOCKED` makes a concurrent transaction **step over** rows another transaction holds and take the next available ones instead of queueing behind them. Four consequences follow, and they are what make this better than the alternatives in §6.1: no two users can receive the same ticket; no lock contention, so latency does not degrade as concurrency rises; no deadlock is possible, because nothing ever waits; and no external coordination is required.

Avoiding the hot-head problem

A subtlety that appears only under sustained load: if candidates are taken in `id` order, every already-reserved row stays at the front of that order and each new allocation walks past all of them. Measured after just 713 reservations, one allocate already scanned 68,453 rows and took 145 ms, and it kept worsening.

The fix is to pick a **random concrete number inside the pattern** — `****23` becomes, say, `481723` — and do a point lookup, retrying up to three times before falling back to a pattern scan. Concurrent users spread themselves across $10^{\text{wildcards}}$ numbers, so collisions are rare and the cost is constant regardless of how much inventory has gone. `SKIP LOCKED` still provides correctness on both paths; the random probe is purely a performance measure.

Reservation expiry

A reservation that is never completed must not hold inventory forever. A background sweep releases anything reserved longer than five minutes. In production this belongs in a scheduled job rather than the API process, so it runs once per interval regardless of replica count.

The measured proof that this works — 351,481 allocations under contention with zero duplicates — is in §8.1. Recorded as ADR-003 and ADR-004.

06 ALTERNATIVES CONSIDERED

What else was measured, and why it lost

6.1 Storage and coordination

REQUIREMENT	WHY POSTGRESQL ANSWERS IT
Duplicate-free distribution	`FOR UPDATE SKIP LOCKED` is a native, atomic, deadlock-free work-distribution primitive. This is the deciding factor: the hardest requirement in the challenge is a one-line feature of the database.
Wildcard search performance	Expression indexes, partial indexes and a cost-based planner allow the 1M-number space to be indexed several ways. Measured p95 of 56 ms at 10M rows under 100 concurrent users.

REQUIREMENT	WHY POSTGRESQL ANSWERS IT
Transactional integrity	Reserving a ticket and writing the audit ledger are the same transaction. With a separate lock service they could not be.
Operational simplicity	One component to deploy, back up, monitor and reason about. Total storage for 10M tickets and all indexes is roughly 1 GB.
Scaling headroom	The measured cost model in §7.2 shows one instance serving 10,000 requests/sec at 39 % of a six-core allocation.

Alternatives considered and rejected

OPTION	WHY NOT
Redis as a cache in front	Adds a component and a failure mode, and creates a consistency problem exactly where correctness matters most. Allocation must write to the database anyway, so the cache cannot own the decision.
Redis distributed lock	Requires managing lock expiry and the process that dies holding a lock, and still writes to the database. `SKIP LOCKED` gives the same guarantee transactionally and for free.
`SELECT FOR UPDATE` without `SKIP LOCKED`	Correct but serializing: every allocator queues behind the first, and latency degrades to seconds as soon as users contend.
Optimistic locking with a version column	Under heavy contention on a popular pattern the retry rate climbs until throughput collapses.
Application-level in-memory lock	Cannot work across replicas, so it fails the moment the API scales out — which §7.2 shows is the primary scaling path.
Elasticsearch or a search engine	Solves a text-search problem this is not. Six digits is a tiny fixed key space, and it offers nothing for the atomic-allocation requirement, which is the actual difficulty.
Precomputing every pattern's results	10^6 possible patterns and result sets up to a million rows. Storage and invalidation cost dwarf the query cost being avoided.

Recorded as ADR-001 and ADR-003.

6.2 Index strategies

Reporting a single strategy without alternatives would establish only that it works, not that it is good. Four indexing strategies were implemented and measured on the same 10M-row dataset.

STRATEGY	WHAT IT DOES	INDEX SIZE
0 Baseline	`lpad(number::text,6,'0') LIKE '____23'`	none usable
A Expression indexes	One partial index per digit on `tickets`, relying on the planner to combine them	**397 MB**
B Dimension table	Resolve the pattern on `numbers`, then join	192 MB
C B + prefix range	As B, but a fixed leading run becomes a key range and an exact pattern a point lookup	**192 MB**

Single-query cost (milliseconds)

PATTERN	WILDCARDS	0	A	B	C
`123456`	0	**955**	**163**	5.5	**0.06**
`123***`	3	2.6	19.9	3.0	**0.12**
`****23`	4	0.28	0.88	3.4	**0.13**
`**3***`	5	0.05	0.03	1.1	**0.20**
`******`	6	0.03	0.02	0.02	**0.02**

The baseline looks competitive on the lower rows only because `LIMIT 20` lets a scan stop early when matches are abundant. The exact-match row is the honest test — about ten matching tickets in ten million — and there the chosen design is **roughly 15,000× faster**.

Under load: 10 → 100 concurrent users, one 500m-CPU API pod

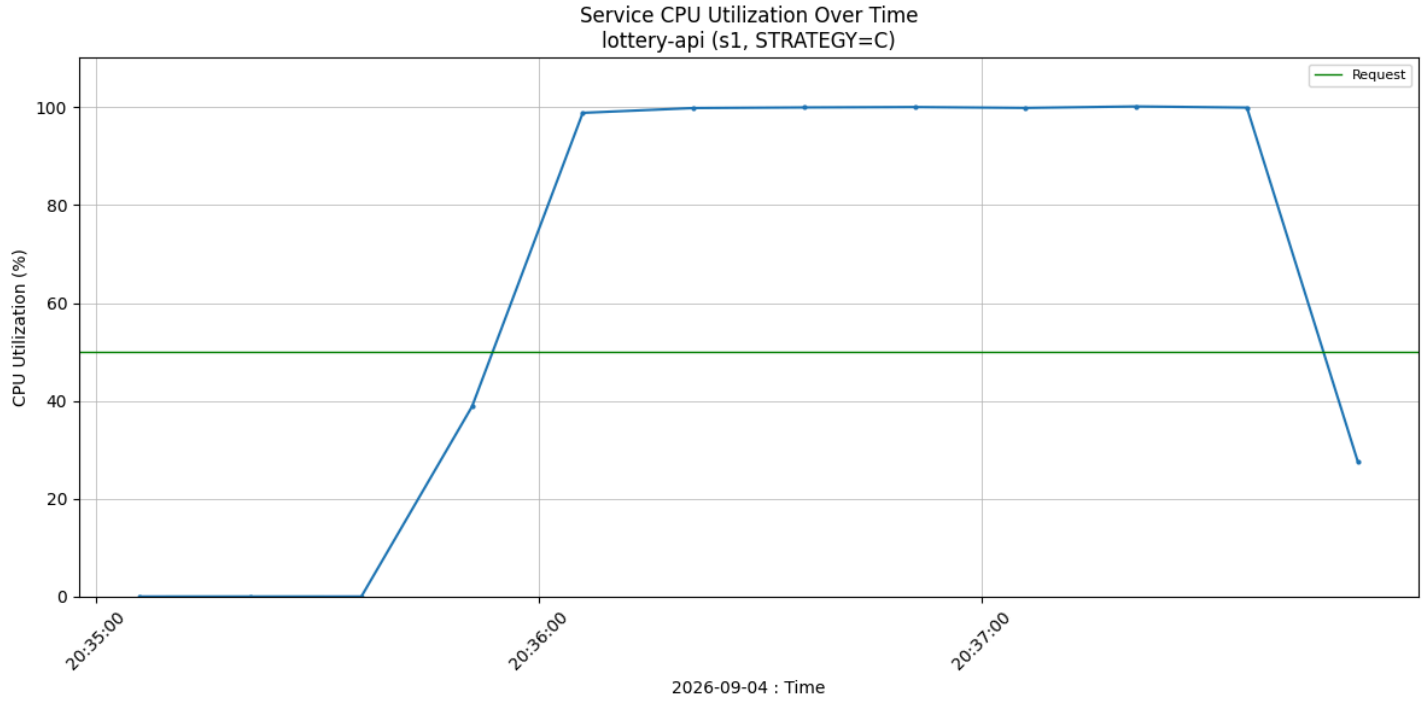
STRATEGY	P95 W0	P95 W3	P95 W4	P95 W5	P95 ALL	REQ/S	FAILED
C	56.3	55.9	55.5	55.6	**55.8**	**5,346**	0 %
B	147.1	144.6	108.4	108.7	142.3	926	0 %
A	6,718	5,162	4,859	4,860	**5,972**	**22**	0 %
0	1,053	1,021	1,023	1,021	1,022	145	**95.7 %**

Latency is flat across wildcard classes under Strategy C, which is the property §2.1 argued was necessary.

Where each strategy spends its resources

STRATEGY	API CPU	POSTGRES CPU	POSTGRES MEMORY FREE
C	**100 % — at limit**	88 %	203 MiB
B	24 %	**101 % — at limit**	279 MiB
A	1 %	**101 % — at limit**	**8 MiB**
0	1 %	**102 % — at limit**	568 MiB

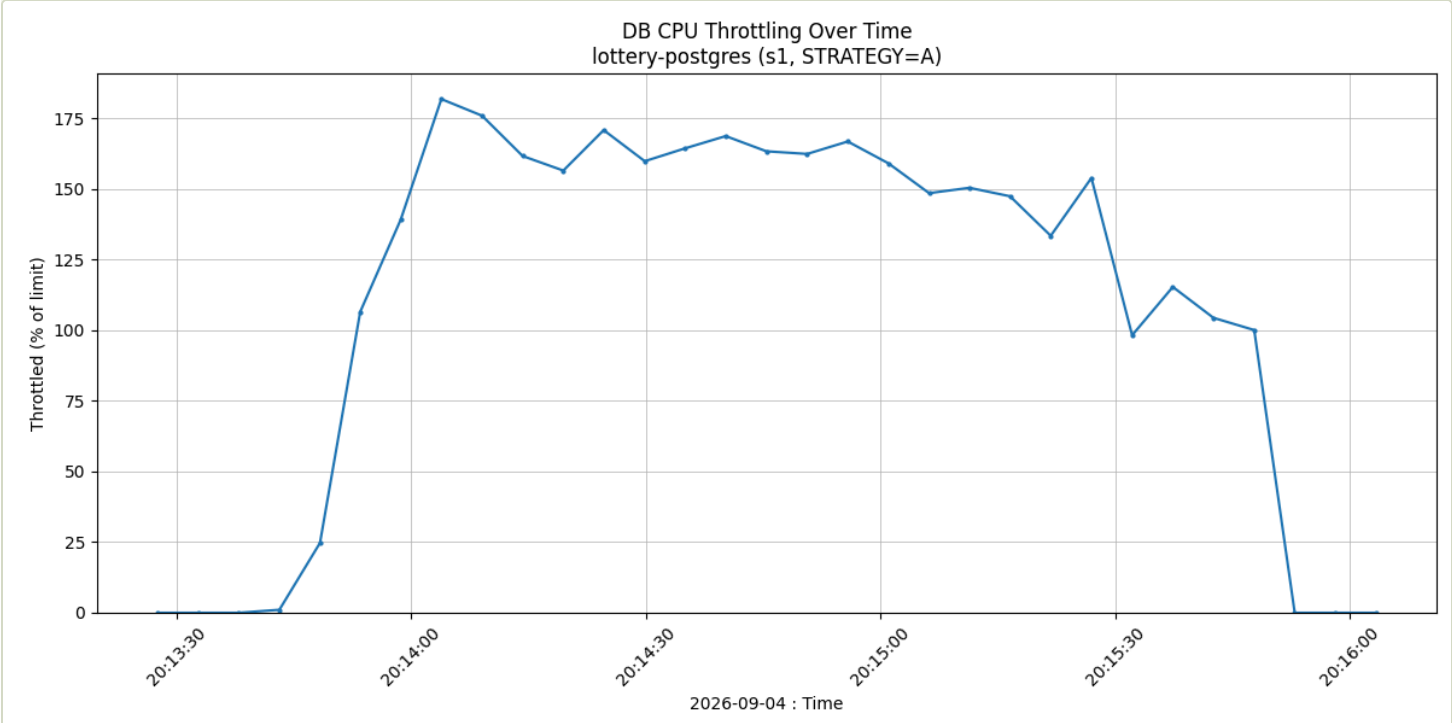
This table matters more than the latency table for a design decision. **Strategy C is the only one that moves the bottleneck out of the database.** Postgres retains headroom, so throughput can be raised by adding stateless API replicas, which is cheap and effectively unlimited. Under B and A the database is the constraint and no amount of API scaling helps.



Under Strategy C the API pod saturates first. That is the desirable shape: the constrained resource is the one that can be replicated. The green line is the pod's CPU request.

Why Strategy A fails, and why that is useful

Strategy A is the intuitive answer: index every digit and let the planner intersect them. It is the worst option measured, and understanding why is what justifies rejecting it. A single digit is only 10 % selective — an index on it still identifies a million rows out of ten million. Combining six such indexes costs more than it saves, so the planner uses one and filters with the rest.



Strategy A had the database denied up to 182 % of its CPU allocation per second. Free memory simultaneously fell to 8 MiB of 1 GiB, because 397 MB of indexes do not fit alongside the working set. The lesson generalizes: index size and index *selectivity* are different things, and an index that fails to eliminate most rows can cost more than a sequential scan.

Recorded as ADR-002.

6.3 Strategy D — measured and rejected

PostgreSQL 17 can satisfy `= ANY(array)` in a single B-tree descent, which makes a different query shape viable: let the API enumerate the matching numbers and send them as an `int[]` instead of resolving them with a subquery against `numbers`. Call it **Strategy D**, applicable only to patterns with at most three wildcards — 1,000 values, roughly 4 KB — with anything wider falling back to C.

Measured with a dedicated script whose classes include `dd*d**`, where the fixed digits are *not* a leading run and Strategy C therefore has no key range to exploit (`pg16-d1-*/`, `pg17-d1-*/`):

VERSION	STRATEGY	P95	THROUGHPUT	API CPU	DB CPU
PG16	C	58.3 ms	5,023 req/s	100 %	72 %
PG16	**D**	39.1 ms	**6,051 req/s**	100 %	**37 %**
PG17	C	38.8 ms	4,982 req/s	100 %	73 %
PG17	**D**	**33.7 ms**	**6,047 req/s**	100 %	**37 %**

The result contradicted the prediction that motivated the test. Strategy D was expected to need 17's optimization to pay off; instead it halves database CPU on *both* versions, because the saving comes mostly from dropping the join against `numbers` rather than from the new index scan. Two further readings matter more than the headline: **upgrading buys the same latency as writing Strategy D** — C on 17 (38.8 ms) is indistinguishable from D on 16 (39.1 ms), and one of those costs a version bump while the other costs a second predicate path in the code; and **what D actually produces is database headroom, not speed**, since its throughput is identical on both versions because at 37 % database CPU it is bound by the API, and a faster database cannot help a workload that is not waiting on one.

Strategy D is therefore not adopted. §7.2 puts the database at 39 % of its allocation at 10,000 requests/sec, so more database headroom solves a problem this system does not have, while D adds a second code path, a per-request allocation of up to 1,000 integers, and 4 KB of extra traffic on every narrow query. It is the right answer only if the database later becomes the constraint, and this measurement says it would then roughly halve database CPU.

6.4 PostgreSQL version: measured on 16, recommend 17

Every figure in §6.2 was taken on PostgreSQL 16.13, which is why the tables say 16. The suite was then re-run on 17.11 under identical conditions — same 10M-row dataset, same baseline pod profile of one 500m API pod against a 2000m/1Gi database,

same scripts — so the two are directly comparable.

LOAD TEST	PG 16.13	PG 17.11	CHANGE	EVIDENCE
Strategy C search, p95	55.8 ms	**41.9 ms**	**−25 %**	`s1-C/` → `pg17-s1-C/`
Strategy C search, max	142 ms	**86 ms**	**−39 %**	”
Strategy C search, throughput	5,346 req/s	**5,952 req/s**	**+11 %**	”
Allocation under 200-VU contention, p95	73.8 ms	**68.0 ms**	**−8 %**	`s2b/` → `pg17-s2b/`
Allocation throughput	4,258/s	**4,419/s**	**+4 %**	”
Duplicate tickets under contention	**0 of 255,677**	**0 of 265,344**	unchanged	`30-verify.txt`
Baseline `LIKE`, requests failed	**95.7 %**	**96.3 %**	unchanged	`s1-0/` → `pg17-s1-0/`

Deploy on 17. Search gets 25 % better latency and 11 % more throughput for no code change, and the duplicate-free guarantee is untouched — 265,344 allocations under 200-way contention on 17 produced zero duplicates, exactly as on 16, because the guarantee rests on `SKIP LOCKED` rather than on anything version-specific.

It changes nothing structural. The strategy ranking is identical on both versions and the baseline still fails 96 % of requests under 100 concurrent users. PostgreSQL 17 made individual baseline queries up to 83 % faster, and that made no difference at all to its behaviour under load — a concrete reminder that a single-query benchmark and a concurrency benchmark answer different questions. One claim in §6.2 must be softened accordingly: the exact-pattern advantage over the baseline measures 17,000× on 16 and 11,700× on 17, because 17 speeds the baseline up more than it speeds up Strategy C. Both come from single `EXPLAIN` runs and vary between runs, so the defensible statement is **four orders of magnitude on either version**.

The figures in §6.2 are kept as measured on 16 rather than restated for 17, so that every number in this document has one provenance. Treat them as the conservative case. Recorded as ADR-007.

07 CROSS-CUTTING CONCERNS

Performance tradeoffs, capacity, operations, observability

7.1 Performance tradeoffs accepted

- **An extra join.** Strategy C reads two tables instead of one. Measured cost: fractions of a millisecond, because the number table stays entirely in cache.
- **An extra table to seed.** One statement, run once.
- **Leading wildcards skip the range optimization.** `****23` falls back to per-digit indexes — still 0.13 ms, so the fallback is not a weak path.
- **Allocation degrades near exhaustion.** With `****23` 96 % drained, p95 rose from 74 ms to 548 ms as random probes increasingly missed. This is a product problem — stop offering a nearly sold-out pattern — rather than a database one, but it must be designed for.

7.2 Capacity and scaling

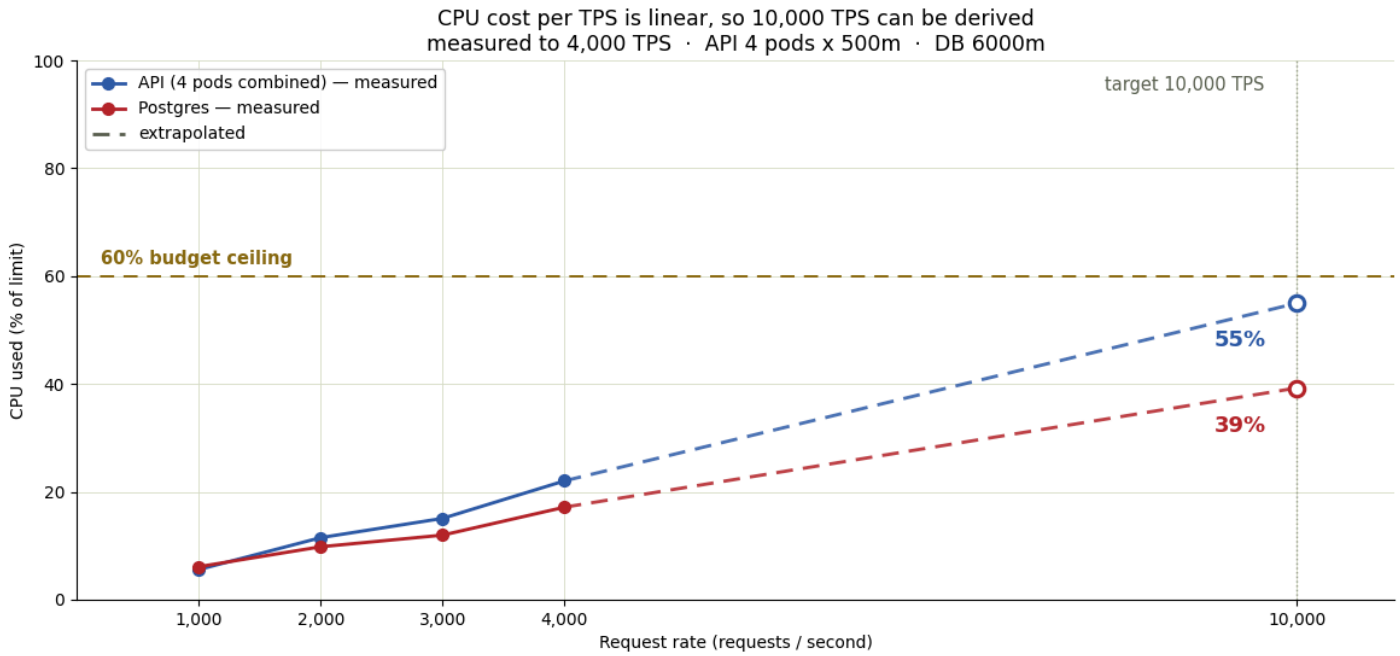
A stepped load test at 1,000 / 2,000 / 3,000 / 4,000 requests per second separates fixed background work from marginal per-request cost.

```
CPU used  =  background  +  cost per request × rate

API       =    1.9 m-core  +  0.1098 m-core per req/s
Postgres  =  146.1 m-core  +  0.2209 m-core per req/s
```

Applying that model to a target of 10,000 requests/sec, holding both components under 60 % of their limits:

COMPONENT	REPLICAS	CPU LIMIT	MEMORY LIMIT	PROJECTED AT 10,000 REQ/S
API stateless — scale out	4	500m per pod	128Mi per pod	**55 % CPU · 14 % memory**
PostgreSQL single instance	1	6,000m	3Gi	**39 % CPU · 49 % memory**



Solid lines are measured, dashed are derived. The horizontal line is the 60 % budget. At 10,000 requests/sec both components remain below it. Plotting percentage-of-limit rather than raw CPU lets two differently sized components share one budget line.

Three points a purely CPU-based model would miss. **Database memory does not grow linearly with load** — it grows with the hot data pulled into cache and then flattens, with per-step increments of 256 → 103 → 66 MiB converging near 1,510 MiB; extrapolating it linearly would have oversized the instance by more than half a gigabyte. **Connection pooling is not a constraint**: required concurrency is rate × service time = 10,000 × 1.19 ms ≈ 12 connections against 40 provisioned. And **the API must be scaled out, not up**, because the Go runtime is pinned to one processor per pod, so a larger pod does not help while more pods do — and they add fault tolerance.

Beyond roughly 20,000 requests/sec the single database becomes the limit. The next step there is read replicas for search only, with allocation continuing to use the primary, because `SKIP LOCKED` requires the authoritative copy.

7.3 Reliability and operations

Liveness must not depend on the database. Under the baseline strategy the connection pool filled with slow queries, the readiness probe timed out, Kubernetes removed the pod from the Service, and 19,134 of 19,990 requests failed. That is correct behaviour — but had the *liveness* probe also queried the database, Kubernetes would have killed and restarted healthy pods during a database slowdown, turning a degradation into an outage. Hence `/healthz` checks the process only, while `/readyz` pings the database.

Resource limits must be communicated to the runtime. A Go process reads the host's CPU count, not its cgroup quota, unless told otherwise. `GOMAXPROCS` and `GOMEMLIMIT` are set explicitly; without them the runtime creates far too many threads and is eventually OOM-killed.

Watch CPU throttling, not just CPU usage. Throttling is invisible in average CPU graphs and was the mechanism behind Strategy A's collapse. Alongside it, monitor query plans through `pg_stat_statements`, lock contention through `pg_locks`, and per-pod usage against limits.

Stability was verified with a 10-minute soak of mixed search, allocate and release traffic: 743,994 requests, zero errors, zero restarts, no OOM kills, and API memory flat at 27 MiB of a 128 MiB limit.

7.4 Observability

Query plans via `pg_stat_statements`; lock contention via `pg_locks` and `pg_stat_activity`; per-pod CPU and memory against limits; and CPU throttling, which is invisible in average CPU graphs but was the mechanism behind Strategy A's collapse (§6.2). A full observability stack is a non-goal (§3.2); these are the signals it would need to carry.

08 VALIDATION AND EVIDENCE

Every claim, and where to verify it

8.1 Load-test evidence for duplicate-free distribution

Two contention runs, each 200 concurrent users hammering a single pattern for 60 seconds.

	`****23` ~100K TICKETS	`**3***` ~1M TICKETS
Allocations in 60 s	95,804	255,677
Ledger rows = reserved tickets = distinct ids	95,804	255,677
Tickets issued twice	0	0
Ledger rows disagreeing with ticket state	0	0
HTTP errors	0	0

351,481 allocations, zero duplicates. The audit is one query that must return no rows: `SELECT ticket_id, count(*) FROM allocations GROUP BY ticket_id HAVING count(*) > 1;`

8.2 Automated tests

The same invariant is asserted in `go test` against a real PostgreSQL: 50 goroutines competing for 120 tickets, demand exceeding supply, every ticket issued exactly once, and an exhausted pool returning an empty result rather than an error. The suite runs against every index strategy, including the rejected ones, so the correctness guarantee is shown to be independent of the search path.

8.3 Evidence index

CLAIM	SOURCE
Query plans and index sizes	`docs/explain-k8s-10M.txt`
Cost of the rejected `ORDER BY` design	`docs/explain-k8s-10M-orderby.txt`
Strategy comparison under load	`docs/evidence/s1-C/` , `s1-B/` , `s1-A/` , `s1-0/`

CLAIM	SOURCE
Zero duplicate distribution	`docs/evidence/s2/30-verify.txt` , `s2b/30-verify.txt`
Stability over 10 minutes	`docs/evidence/s3/`
Capacity model for 10,000 req/s	`docs/evidence/c1-capacity/50-capacity.json`
PostgreSQL 16 vs 17 query plans	`docs/evidence/pg16/explain.txt` , `pg17/explain.txt`
PostgreSQL 17 under load	`docs/evidence/pg17-s1-C/` , `pg17-s1-0/` , `pg17-s2b/`
Strategy D on both versions	`docs/evidence/pg16-d1-C/` , `pg16-d1-D/` , `pg17-d1-C/` , `pg17-d1-D/`
Full analysis of every run	`docs/RESULTS.md`

Each evidence bundle contains the complete load-generator log, the summary as JSON and HTML, resource charts, the raw samples those charts were drawn from, and the SQL verification output. All of it is reproducible with a single command.

09 RISKS AND LIMITATIONS

What this design has not yet proven

- **10,000 req/s is calculated, not observed.** The highest measured rate is 4,000 req/s; the target is a 2.5× linear extrapolation. Four data points support linearity but do not prove it holds further out. Validate with a load generator on separate hardware before committing.
- **Near-exhaustion behaviour is only partly characterized.** Allocation latency is known to degrade when a pattern is nearly sold out, but the threshold at which a pattern should be withdrawn from sale has not been determined.
- **All measurements come from a single node**, where API, database and load generator share one CPU pool. Absolute throughput on separated hardware would differ; the comparison between strategies, which is what the recommendation rests on, would not.

- **Failure modes are untested.** Nothing here covers pod loss, rolling updates or database failover. Capacity is reduced during those events, so provision at least one replica beyond the computed minimum.
- **A skewed inventory distribution is untested.** All measurements start from a fully available pool. With 80 % of tickets already sold the partial index shrinks but the random-probe hit rate falls; both effects need measuring.
- **Out of scope by design**: authentication, payment, draw scheduling, replication and any caching layer.

10 APPENDIX

Decision records, glossary, and the scoring map

A. Architecture decision records

One record per decision, in `adr/`. Each states the context, the decision, the alternatives rejected, the consequences accepted, and the evidence bundle that supports it.

ADR	DECISION	SECTION
`ADR-001`	Use a single PostgreSQL instance; no cache, no lock service	§6.1
`ADR-002`	Resolve patterns on a 1M-row `numbers` table, then join (Strategy C)	§5.3, §6.2
`ADR-003`	Guarantee duplicate-free allocation with `FOR UPDATE SKIP LOCKED` in one statement	§5.4
`ADR-004`	Allocate by random concrete-number probe before falling back to a pattern scan	§5.4
`ADR-005`	No `ORDER BY` on search	§5.3
`ADR-006`	Liveness must not touch the database; readiness must	§7.3
`ADR-007`	Deploy on PostgreSQL 17	§6.4
`ADR-008`	Do not adopt Strategy D	§6.3

B. Glossary

TERM	MEANING IN THIS DOCUMENT
p95	The latency that 95 % of requests were faster than. Used instead of the mean because the mean hides the slow tail users actually feel.
VU	Virtual user — one simulated client in the load generator, issuing requests in a loop. 200 VUs means 200 concurrent clients.
wildcard class	Patterns grouped by how many `*` they contain (w0, w3, w4, w5). Latency is reported per class because result-set size differs by six orders of magnitude between them.
`SKIP LOCKED`	Option on `SELECT … FOR UPDATE` that makes a transaction skip rows another transaction holds instead of waiting for them. The basis of the duplicate-free guarantee.
BitmapAnd	The planner's way of intersecting several indexes. Pays off only when each index is highly selective; a single-digit index is 10 % selective, which is why Strategy A fails.
partial index	An index restricted by a `WHERE` clause — here `WHERE status = 0` — so it covers only available tickets and shrinks as inventory sells.
working set	Memory in use excluding reclaimable page cache. Raw usage of a PostgreSQL container sits near 100 % of its limit at all times because it fills the cgroup with cache; the working set is the honest number.
CPU throttling	Time a container was denied CPU because it exceeded its quota. Invisible in average CPU graphs; measured directly from the cgroup.
liveness / readiness	Liveness asks whether the process is alive; failing it restarts the pod. Readiness asks whether the pod can take traffic; failing it removes the pod from the load balancer without restarting it.

C. Mapping to the challenge's evaluation criteria

CRITERION	WHERE IT IS ADDRESSED
Feasibility	§5.1 architecture, §5.2 data model, §5.3 algorithm — a working implementation was built and measured
Performance	§6.2 four-way comparison at 10M rows, §7.2 capacity model
Correctness	§5.4 concurrency design and the 351,481-allocation audit
Real-world practicality	§6.1 database choice and rejected alternatives, §7.3 operations, §9 limitations
Creativity	§2.1 the one-million search space, §5.3 collapsing a fixed prefix to a key range, §5.4 the random-probe allocation path

Design document for the Lottery Search System challenge. The markdown source is `docs/DESIGN_DOCUMENT.md`; supporting analysis is in `docs/RESULTS.md` and the capacity plan, and all raw measurements are under `docs/evidence/`.